Project Charter

Project Name: Enterprise RAG Knowledge Assistant

Group 11
Ruolin Wang, Yunting Ma, Yen-Jui Liang

MSCS 3999 Capstone
Fall 2026
Clark University

Document Control

Change Record

Date	Author	Version	Change Reference

Reviewers

Sign Off Date	Reviewer	Position	Sign Off

Distribution

Copy Number	Name

Table of Contents

1 PROJECT BACKGROUND 4

1.1 Problem/Opportunity Description 4

1.2 Benefits 4

1.3 Goals 4

1.4 Stakeholders and Clients 4

2 PROJECT SCOPE 5

2.1 Objectives 5

2.2 Deliverables 5

2.3 Out of Scope 6

3 PROJECT PLAN 7

3.1 Approach and Methodology 7

3.2 Project Timeline 7

3.3 Success Criteria 7

3.4 Issues and Policy Implications 7

3.5 Risk Management Plan 8

3.6 Service Transition 8

3.7 Options Analysis 8

4 TECHNICAL FEATURES 10

5 PROJECT ORGANIZATION AND STAFFING 10

6 PROJECT BUDGET 11

7 APPENDIX A- ADDITIONAL INFORMATION 12

1 Project Background

1.1 Problem/Opportunity Description

Modern enterprises accumulate vast quantities of unstructured knowledge spread across multi-format documents, including PDFs, Markdown files, Word documents, and financial reports. Traditional keyword search mechanisms often fail to deliver precise semantic answers, forcing employees to spend excessive time manually searching through long documents. Conversely, standard public Large Language Model (LLM) solutions lack granular security controls, risk exposing sensitive internal corporate data across department boundaries, and are prone to hallucinations without verifiable context grounding.

1.2 Benefits

Productivity & Efficiency: Drastically reduces information discovery time for internal teams by generating direct, contextual answers to complex queries.

Corporate Data Governance: Enforces strict Role-Based Access Control (RBAC) at the retrieval level, preventing unauthorized internal access to restricted document segments.

Auditability & Trust: Delivers interactive, page- and line-level citation highlights, allowing enterprise users to instantly verify AI-generated answers against primary sources.

Cost & Risk Avoidance: Mitigates data leakage risks associated with unmanaged public LLM usage while reducing redundant technical support and internal research overhead.

1.3 Goals

The goal of this project is to design, implement, and evaluate a functional prototype of a secure, enterprise-oriented Retrieval-Augmented Generation (RAG) knowledge assistant.

The system will enable authorized users to search for internal documents, retrieve relevant information, and receive grounded answers with verifiable source citations.

The project will focus on three core capabilities:

1. **Accurate Information Retrieval:** Integrating keyword-based and semantic retrieval to improve access to relevant document content.
2. **Secure Knowledge Access:** Implementing Role-Based Access Control (RBAC) throughout document retrieval and answer generation.
3. **Verifiable AI Responses:** Providing source-linked citations that allow users to inspect the evidence supporting generated answers.

The project will evaluate retrieval quality, access-control enforcement, citation accuracy, and system responsiveness within a defined prototype environment

1.4 Stakeholders and Clients

Primary Clients: Enterprise knowledge workers, internal legal/finance teams, and corporate department heads requiring rapid document search.

Internal Stakeholders: Information Security Officers, Systems Administrators, and IT Compliance Teams responsible for data governance.

Academic Sponsors & Advisors: Clark University Department of Computer Science faculty and Capstone reviewers.

2 Project Scope

2.1 Objectives

Requirements Gathering & Data Security Architecture: Define enterprise RBAC clearance levels, document ingestion policies, and data classification schemas.

Multi-Format Ingestion & Hybrid Retrieval Engine: Build a backend pipeline integrating layout-aware document parsing, sparse (BM25) search, dense vector embeddings, and Qdrant vector storage with metadata security filters.

Advanced RAG & Reranking Pipeline: Implement a cross-encoder reranking stage and hallucination-resistant LLM generation module configured with strict context-bounding prompts.

Full-Stack Web Dashboard & Citation Interface: Develop a responsive Web dashboard offering dynamic document uploading, user authentication/authorization, conversational search, and interactive page/line citation highlighting.

System Evaluation & Security Audit: Conduct end-to-end benchmarking assessing retrieval accuracy (Precision/Recall/MRR), latency, hallucination rates, and RBAC security enforcement.

2.2 Deliverables

Objective 1 – Requirements Gathering & Data Security Architecture

Project Deliverable	Work Products/Description
Architecture Specification	Architecture diagram, RBAC matrix, API contract, data model

Objective 2 – Multi-Format Ingestion & Hybrid Retrieval Engine

Project Deliverable	Work Products/Description
Document Processing Module	PDF/Markdown parsers, chunking, metadata extraction, hybrid search

Objective 3 – Advanced RAG & Reranking Pipeline

Project Deliverable	Work Products/Description
Answer Generation Module	Grounded response generation, citation mapping, optional reranking

Objective 4 – Full-Stack Web Dashboard & Citation Interface

Project Deliverable	Work Products/Description
Functional Web Application	Authentication, upload, chat, document management, citation preview

Objective 5 – System Evaluation & Security Audit

Project Deliverable	Work Products/Description
Evaluation Report	Labeled test set, retrieval metrics, latency results, RBAC test cases

2.3 Out of Scope

Custom LLM Pre-training: Training foundational base models from scratch is excluded due to computational constraints; existing open/hosted foundation models will be leveraged.

Legacy Mainframe Integrations: Direct native integration with proprietary, non-standard enterprise legacy databases (e.g., AS400).

Real-Time Voice/Audio Processing: Support for speech-to-text or real-time audio conversation modes is deferred to future platform iterations.

On-Premise Physical Hardware Provisioning: Deploying specialized physical server infrastructure; all hosting will utilize containerized cloud/virtual environments.

Scope Prioritization

The project will follow a phased implementation strategy to ensure the delivery of a complete and testable system within the Capstone timeline.

Minimum Viable Product (MVP):

The MVP will support PDF and Markdown ingestion, role-based document access, hybrid retrieval, grounded question answering, page-level source citations, and a functional Web interface.

Secondary Features:

DOCX ingestion, cross-encoder reranking, and additional document management features will be implemented after the core workflow is operational.

Stretch Goals:

Advanced multi-column and table parsing, precise line-level citation highlighting, and fine-grained chunk-level access control will be considered based on the team's remaining development capacity.

The MVP will take priority over optional functionality to ensure that the final submission includes an integrated, deployable, and evaluated application.

2.4 Approach and Methodology

The project will follow an Agile development methodology with two-week iterations and weekly integration checkpoints.

The system will adopt a modular full-stack architecture consisting of a React/TypeScript frontend, a Python FastAPI backend, a Qdrant vector database, and supporting document processing and retrieval modules.

Backend components will be organized into clearly defined modules for document ingestion, retrieval, authorization, and answer generation.

Docker Compose will be used to simplify local development, service configuration, and deployment.

The team will prioritize early end-to-end integration, automated testing, and incremental feature delivery.

2.5 Project Timeline

ID	Task Name	Start	Finish	Duration
1	Architecture & Security Requirements Definition	09/16/2026	09/30/2026	2 Weeks
2	Ingestion & Layout-Aware Document Parsing Pipeline	10/01/2026	10/14/2026	2 Weeks
3	Hybrid Retrieval (BM25 + Qdrant RBAC Vector Indexing)	10/15/2026	10/28/2026	2 Weeks
4	Cross-Encoder Reranking & LLM Citation Generation Engine	10/29/2026	11/11/2026	2 Weeks

5	Full-Stack Web Dashboard & Citation UI Development	11/12/2026	11/25/2026	2 Weeks
6	System Integration, RBAC Security Audit & Evaluation	11/26/2026	12/05/2026	1.5 Weeks
7	Capstone Final Documentation & Handoff	12/06/2026	12/15/2026	1.5 Weeks

2.6 Success Criteria

The project will be evaluated using a predefined benchmark dataset containing representative enterprise documents, labeled queries, and simulated user roles.

1. Access Control

The system must prevent unauthorized document content from appearing in retrieval results, generated answers, and citation previews across all predefined security test cases.

2. Retrieval Quality

Retrieval performance will be evaluated using Recall@5 and MRR@10 against manually labeled relevant documents or passages.

The initial target is MRR@10 $\geq$ 0.70. Results from dense-only, hybrid, and reranked retrieval configurations will be compared.

3. Citation Accuracy

At least 90% of evaluated citations should correctly identify the source document and supporting page or passage.

4. System Responsiveness

The system will report median and 95th-percentile response latency, distinguishing time to first token from complete answer generation.

An initial target of p95 time to first token $\leq$5 seconds will be assessed under a documented test environment.

5. Answer Quality

The evaluation will measure answer correctness, evidence support, and the system's ability to decline answering when relevant authorized information is unavailable.

All numerical targets will be finalized following preliminary benchmark testing.

2.7 Issues and Policy Implications

Data Privacy Policies: Data ingestion and chunk storage must strictly align with organizational privacy regulations (e.g., SOC2, GDPR concepts).

API Quota & Rate Limits: Dependence on external API endpoints for embeddings or LLM generation requires token budgeting and fallback strategy implementations.

2.8 Risk Management Plan

Risk Factor	**Probability (H-M-L)**	**Impact (H-M-L)**	**Risk Management Action**
Layout parsing errors on complex multi-column PDFs/tables	M	H	Incorporate dedicated layout-aware parsers (e.g., Unstructured/Marker) with OCR fallback strategies.
nuthorized context leakage across user role levels	L	H	Implement strict server-side vector metadata filtering in Qdrant prior to semantic searching.
High query latency caused by cross-encoder reranking	M	M	Optimize reranker batch sizes and implement caching for frequently accessed document chunks.
Excessive API consumption costs during testing	M	L	Utilize lightweight local embedding models (e.g., HuggingFace sentence-transformers) during initial development phases.

2.9 Service Transition

Support & Maintenance: Detailed system administration documentation, REST API endpoints documentation (OpenAPI/Swagger), and Docker configuration scripts will be delivered.

Transitional Support: Deployment guides will be provided to ensure smooth setup across cloud or locally hosted containerized environments.

Service Management: System operational status checks, health monitoring endpoints (/health), and vector collection backup procedures will be embedded into the platform codebase.

2.10 Options Analysis

Option A: Pure Dense Vector Search vs. Hybrid Search (Chosen): Dense vector search alone struggles with exact keyword matching (e.g., SKU numbers, specific legal codes). Hybrid search (BM25 + Qdrant Dense Vector) was selected to maximize retrieval recall and precision.

Option B: Third-Party Hosted Vector Database vs. Self-Hosted Qdrant (Chosen): Self-hosted/containerized Qdrant was selected over fully managed proprietary services (e.g., Pinecone) to ensure full control over payload metadata filtering and local security boundaries.

Option C: Standard Chunking vs. Layout-Aware Parsing (Chosen): Naive character chunking breaks multi-column PDF layouts and markdown tables. Layout-aware parsing was chosen to maintain visual structural integrity for downstream citations.

3 Technical Features

Multi-Format Layout-Aware Ingestion: Extracts structural content, complex tables, and multi-column text from PDFs, DOCX, and Markdown files.

Hybrid Search Engine: Fuses sparse BM25 keyword matching with dense semantic embeddings (e.g., OpenAI / BGE embeddings).

Metadata-Enforced RBAC in Qdrant: Embeds authorization tags directly within vector payload metadata, enforcing query clearance filters at the database level.

Cross-Encoder Reranker: Re-scores retrieved candidate chunks using a cross-encoder model to maximize context relevance before LLM generation.

Hallucination-Resistant Generation: Utilizes grounded system prompting requiring explicit document context backing.

Interactive Citation UI: Interactive Web front-end supporting line-level text highlighting and PDF page preview links upon clicking answer citations.

ROLE	NAMES & CONTACT INFORMATION	RESPONSIBILITIES	TIME
Executive Sponsor	Capstone Department Chair / Faculty Advisor	Ultimate academic oversight, strategic guidance, scope approval	5%
Project Sponsor	Clark University CS Department	Approves project charter, reviews milestone deliverables	5%
Project Manager & Backend Lead	Ruolin Wang	- FastAPI backend architecture - Hybrid retrieval and Qdrant integration - Authentication and RBAC - API contracts and deployment coordination	40%
Project Team Member	Yen-Jui (Ray) Liang	- Document ingestion and parsing - Chunking and source metadata - Reranking module	25%

		- Benchmark dataset and evaluation scripts	
Project Team Member	Yunting Ma	- React/TypeScript frontend - Chat, document management and upload UI - Interactive citation visualization - Frontend–backend API integration	25%

4 Project Budget

The project will operate under a preliminary total budget cap of $100 for the Capstone development period.

Open-source development tools and local Docker environments will be used whenever possible.

The budget will be allocated to optional cloud hosting and LLM/embedding API usage.

GPU-based compute resources will not be required for the baseline implementation and will only be considered if performance testing demonstrates a need.

The team will monitor API usage and cloud expenditure throughout development to remain within the approved budget.

Budget Item	Description	Budgeted Cost
One-Time Costs		
Open-Source Technical Stack	Deployment of open-source framework (Qdrant Vector DB, FastAPI, React, Docker Dev Containers)	$0
Development Hardware	Team member laptops and GPUs for local development and model debugging	$0
Student Personnel	Student academic engineering effort (Ruolin Wang, Ray Liang, Yunting Ma)	$0
Total One-Time Costs		$0
Ongoing Costs		
Cloud Compute Infrastructure	Staging Cloud VM (AWS EC2 / GCP) for Qdrant Vector DB and backend hosting	$50
LLM & Embedding API Usage	API token budget (OpenAI / Cohere) for dense embeddings and benchmark evaluation	$50
Total Ongoing Costs		**$100**

5 Appendix A- Additional Information

Additional relevant information may also be included in this request. This information will be used to decide if 1) the project request is approved to move forward to the project charter phase and 2) the project charter receives approval as an approved IT project.